

Klasser i C++

```
#include <string>
class Person {
    std::string m_name;
    std::string m_address;
    int m_skonummer;
public:
    void setName(string const &str);
    void setAddress(string const &str);
    string const &name() const;
    string const &address() const;
};
```

Klassdeklaration = interface

Definition av funktioner i klassen = implementation

Klasser – konstruktor - destruktör

- Två specialfunktioner som har att göra med hur klassen internt fungerar – konstruktor och destruktör
 - Konstruktor – metod som efter att objektet skapats, men före access av detta
 - Samma namn som klassen, ingen returtyp, kan finnas överladdade konstruktörer

```
class Person {  
public:  
    Person();                               /*Default konstruktor */  
    Person(string const &str); /* Överladdad */  
}  
  
main() {  
    Person p;                               /* Default används */  
    Person p2("Axel Eklund"); /* Överladdad */  
}
```

Klasser – konstruktor - destruktör

- Destruktor – metod som exekveras just innan objektet slutar att existera
- Samma namn som klassen, med "~" framför. Ingen returtyp, inga argument.

```
class Person {  
public:  
    ~Person();           /* Destruktor */  
}
```

Klasser – konstruktor - destruktör

- Vad görs i konstruktorn / destruktorn
 - Initialisering av datafält
 - Allokering/friställning av minne

```
class Person {  
    Person::Person() {  
        m_name = "";  
    }  
    Person::Person(string const &str) {  
        m_name = str;  
    }  
    Person::~~Person() {  
        m_name="";  
    }  
}
```

Klasser – konstruktor - destruktör

```
/* mystring.h */
class String {
private:
    unsigned int m_len;           /* Stränglängd */
    char *m_str;                 /* Pekare till strängen */
public:
    String() {m_len=0;m_str=0;}   /* Default konstruktor */
    String(char const *str);      /* Överladdad konstr.*/
    ~String()                    /* Destruktor */
};
/* mystring.cpp */
String::String(char const *str) {
    m_len = strlen(str);
    m_str = malloc(sizeof(char)*(m_len+1)); /* Allokera minne */
    strcpy(m_str,str);
}
String::~~String() {
    if (m_str) free(m_str);       /* Frigör allokerat minne */
}
```

Klassmetoder – inline/direkta metoder

- Metoderna kan finnas definierade i klassdeklarationen

```
class String {  
public String() {m_len=0;m_str=0;}  
}
```

- Detta innebär att inget funktionanrop skapas, koden inkluderas "inline"
- Samma kan skapas med "inline"-deklarationen, men då måste impl. finnas i include-filen

```
class String {  
public String();  
}  
  
inline String::String() {  
    m_len =0; m_str = 0;  
}
```


Sammanstatta klasser

- Klasser har ofta i sin tur medlemmar som också är klasser → komposition
- Ex. Person-klassen har datafält av typen string, vilken i sin tur är en klass
- När vi har nästade klasser, hur kommer initialiseringen av de olika klasserna att gå till?
 - Kompilatorn skapar kod som initialiserar behövliga klassar, generar kod för anrop av behövliga konstruktorer

Initialisering av sammansatta klasser

- Då ett objekt skapas, kommer samtidigt objekt av underliggande klasser att skapas, och deras konstruktorer kallas
 - Vi kan explicit definiera hur dessa skall initialiseras
 - Också elementära datatype i strukturer kan initialiseras

```
Person::Person(char const *name, char const
               *address, unsigned skonum)
:
m_name(name),      /* string konstruktor */
m_address(address), /* -"" - */
m_skonummer(skonum) /* init av unsigned *(
{} /* Ingen initialiseringskod */
```

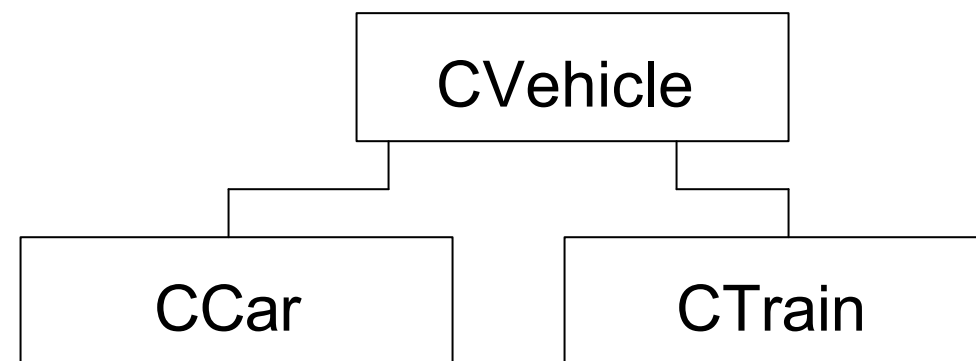

Ärvning

- Objektorienterade språk implementerar i regel ärvning (inheritance). Genom ärvning skapar man en ny klass som ärver förälderns egenskaper

```
class CCar: public CVehicle { /* Dvs ärver CVehicle */
    unsigned m_speed;
public:
    CCar();
    CCar(float w, unsigned speed);
    void SetSpeed(unsigned speed);
};
```

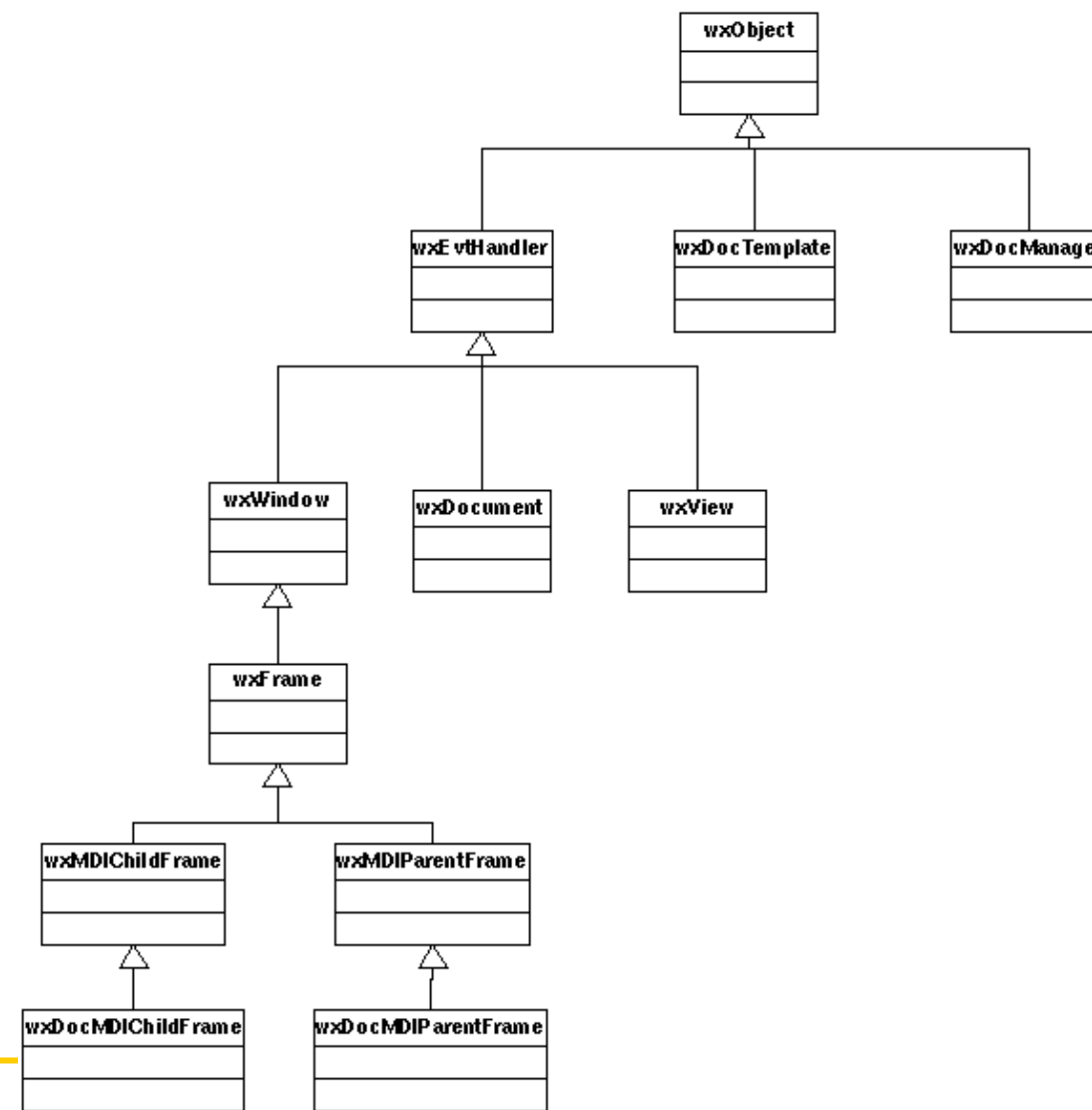
- `": public CVehicle"` genomför ärvningen, så att alla egenskaper hos CVehicle överförs till den nya klassen

Ärvning

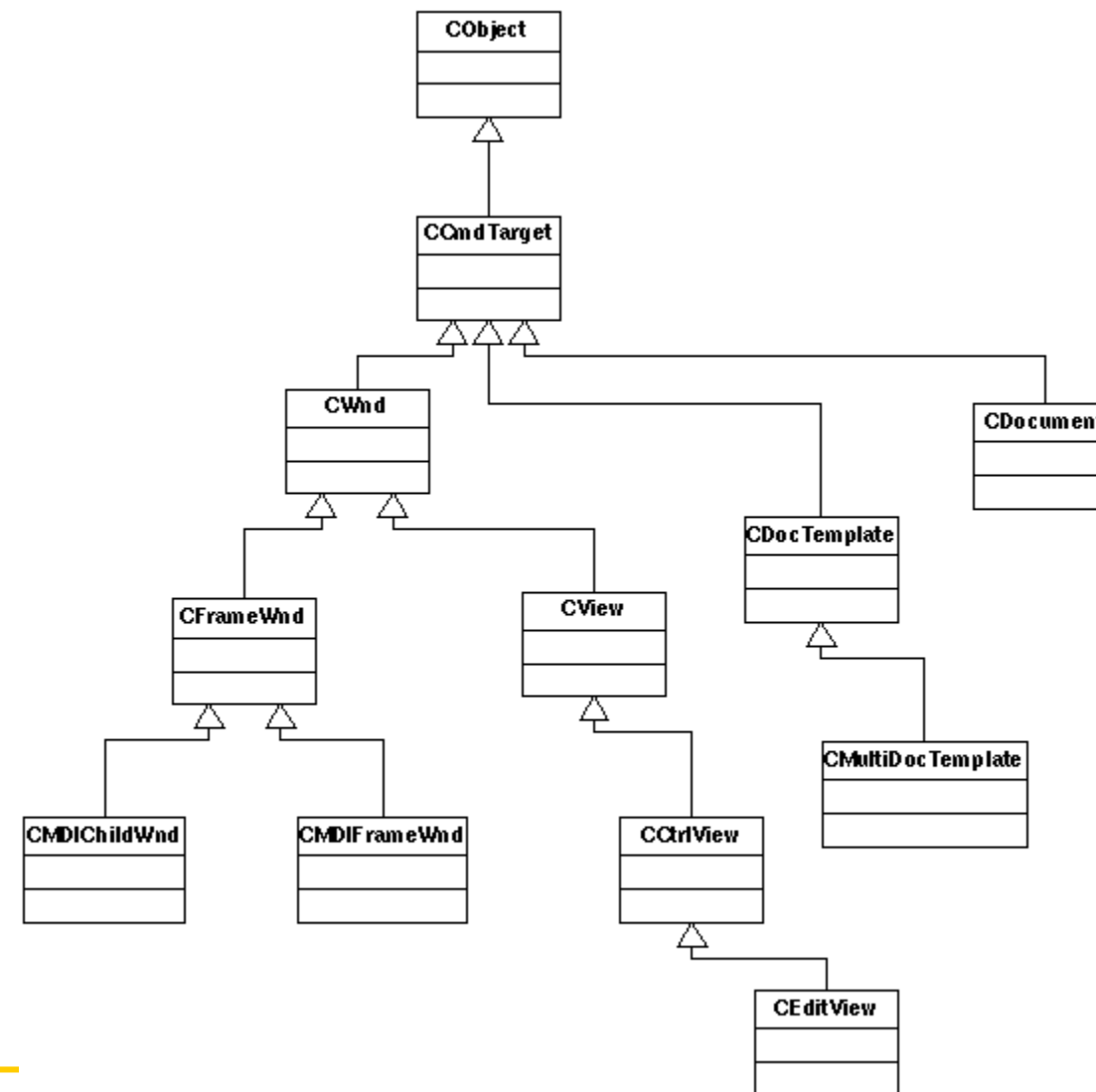


- **Dvs en CCar är en speciell typ av CVehicle**
- **Märk:**
 - **Privata (private:) klassmedlemmar är fortfarande privata, den ärvande klassen måste accessera dessa via funktioner**

wxWidgets klasshierarki



MFC klasshierarki



Konstruktorer för ärvda klasser

- Version "dålig"

```
CCar::CCar() (float weight, unsigned speed) {  
    SetWeight(weight);  
    SetSpeed(speed);  
}
```

- "Dålig", eftersom
 - 1. Default konstruktor för CVehicle anropas, varvid klassmedlemmarnas initialiseras
 - 2. CCar:s konstruktor kallas, varvid klassmedlemmarna IGEN sätts till nytt värde
- Detta borde ske samtidigt, så att endast en konstruktor kallas

Konstruktor för ärvda klasser

- Version "OK"

```
CCar::CCar() (float weight, unsigned speed)
: CVehicle(weight) {
    SetSpeed(speed);
}
```

- Istället kallas basklassens konstruktor nu direkt med motsvarande argument från deriverade klassen

Destruktorer för ärvda klasser

- Då ett objekt frigörs, kommer samtliga destruktorer för de sammansatta klasserna att exekveras

```
class Base {  
    public:  
        ~Base();  
};  
class Derived: public Base {  
    public:  
        ~Derived();  
};  
int main() {  
    Derived derived;  
}
```

I vilken ordning kallas konstruktorer / destruktorer ??

```
// constr.cpp

class Base {
public:
    Base();
    ~Base();
};

class Derived:public Base {
public:
    Derived();
    ~Derived();
};

class Derived2:public Derived
{
public:
    Derived2();
    ~Derived2();
};

Base::Base() {printf("Base class
constructor\n"); }
Base::~~Base() {printf("Base class
destructor\n"); }

Derived::Derived() {printf("Dervied
class constructor\n"); }
Derived::~~Derived() {printf("Derived
class destructor\n"); }

Derived2::Derived2() {printf("Dervied2
class constructor\n"); }
Derived2::~~Derived2() {printf("Derived2
class destructor\n"); }

int main(int argc, char* argv[])
{
    printf("Main starting\n");
    Derived2 der;
    printf("Main ending\n");

    return 0;
}
```

Programmering i C/C++ / JB

I vilken ordning kallas konstruktorer / destruktorer ??

```
% g++ constr.cpp -o constr  
% ./constr
```

```
Main starting  
Base class konstruktor  
Dervied class konstruktor  
Dervied2 class konstruktor  
Main ending  
Derived2 class destruktor  
Derived class destruktor  
Base class destruktor
```

Omdefiniering av medlemsfunktioner

- Funktioner som ingår i basklasser kan bli omdefinierade
 - T.ex. vikt för bil räknas genom att till basvikten räkna till en "beräknad förarvikt"

```
float CCar::GetWeight() {  
    float tmpWeight = CVehicle::GetWeight()+80.0;  
    return tmpWeight;  
}
```

- Om man fortfarande vill komma åt CVehicles gömda funktion, går det genom

```
CVehice::GetWeight(); /* eller för instantierade klass */  
CCar car;  
w = car.CVehicle::GetWeight();
```

Multipel ärvning

- Ärvningen kan också ske från flera klasser, varvid man talar om *multiple inheritance*

```
class CCar: public CVehicle, public CEngine {  
public:  
    CCar();  
    CCar(float w, unsigned speed, unsigned kw);  
}
```

```
CCar::CCar(float w, unsigned speed, unsigned kw):  
    CVehicle(w), CEngine(kw) {  
    SetSpeed(speed);  
}
```

- Istället kallas basklassens konstruktor nu direkt med motsvarande argument från deriverade klassen

Konversion mellan basklass och deriverad klass

- En deriverad klass kan automatiskt också ses som en basklass

```
CCar car(800.0, 180);  
CVehicle veh(40.0);  
veh = car;          /* OK veh subset av car */  
car = veh;          /* Inte OK, hur t.ex avgöra värden för  
                    variabler som saknas i CVehicle ? */  
CVehicle *vp = &car; /* OK, men endast CVehicle's  
                    funktionalitet kan användas */  
CCar *cp = &veh;     /* Inte OK, vi kan inte använda en  
                    pekare till deriverad klass till  
                    att peka på basklass */  
float w=vp->GetWeight(); /* Detta ger dock CVehicle-klassens  
                        version av fordonstyngd (ej +80kg...) */
```

Polymorfism

- Normalt gäller att vi använder funktion enligt pekartyp
 - Men, C++ erbjuder en metod för att använda den deriverade klassens funktion trots att man använder sig av en pekare till basklassen → polymorfism
 - dvs. `vp->GetWeight()` ger 880 istället för 800 i exemplet på förra sidan
 - Genomförs via "late-binding", dvs länkning under exekvering, först då avgörs vilken funktion som verkligen skall anropas

Polymorfism

- Polymorfism, genomförs via direktivet `virtual` för funktionsnamnet. Efter detta kommer den deriverade klassens motsvarande funktion att användas, även om en pekare till basklassen används.

```
class CVehicle {  
public:  
    CVehicle();  
    CVehicle(float w);  
    virtual float GetWeight();  
private:  
    float m_Weight;  
}
```

- Notera, efter att en funktion i basklass har deklarerats *virtual*, kommer den att förbli det i alla deriverade klasser.

Pure virtual functions

- Hittills har vi antagit att den virtuella funktionen också har en implementation i basklassen. Om vi inte har någon implementation i basklassen, talar man om *pure virtual functions*.
 - En pure virtual function skapas genom att till funktionsdeklarationen addera prefixet `virtual` och postfixet `=0`

```
class CVehicle {  
    virtual float GetWeight() = 0;  
}
```
- Vi har deklarerat ett *protokoll*, m.a.o. regler för vad som måste implementeras innan en deriverad klass kan användas

Dynamiskt minnesallokering i C++

- Hittills har vi behandlat statiska eller automatiska klasser
- C++ inför *operatorerna* **new** och **delete** för att allokeras dynamisk minne

```
main() {  
    CVehicle *vp = new CVehicle(12.0);  
    ... // Code  
    delete vp; /* kallar även på destruktorn */  
}
```

- Notera att **new** och **delete** också kallar på konstruktorn/destruktorn

Dynamiskt minnesallokering i C++

- Allokering av räckor
 - `CVehicle *vp = new CVehicle[40];`
 - Explicit allokering
 - Vid allokering av räckor kallas alltid default constructor (konstruktor utan argument)
- Deallokering av räckor
 - Deallokering genom

```
delete [] vp; /* Destruktorer kallas för
                varje objekt skilt */
delete vp;    /* Destruktor endast för
                första dynamiska fältet */
```